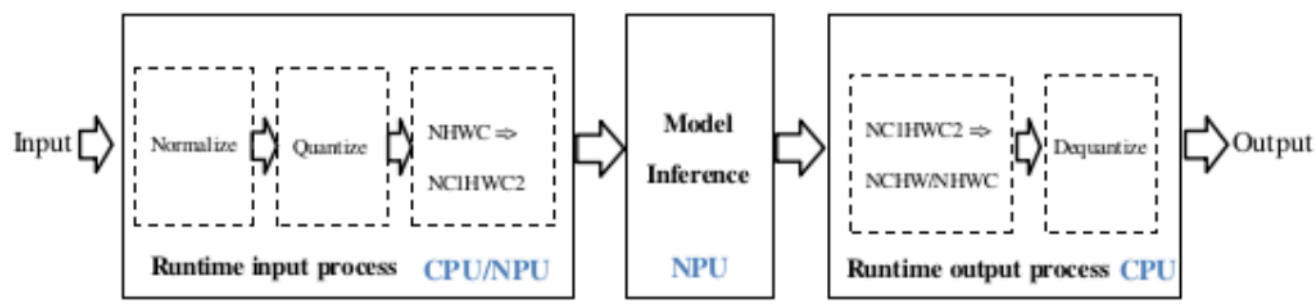


26.RKNPU2—零拷贝C API

1. 零拷贝介绍

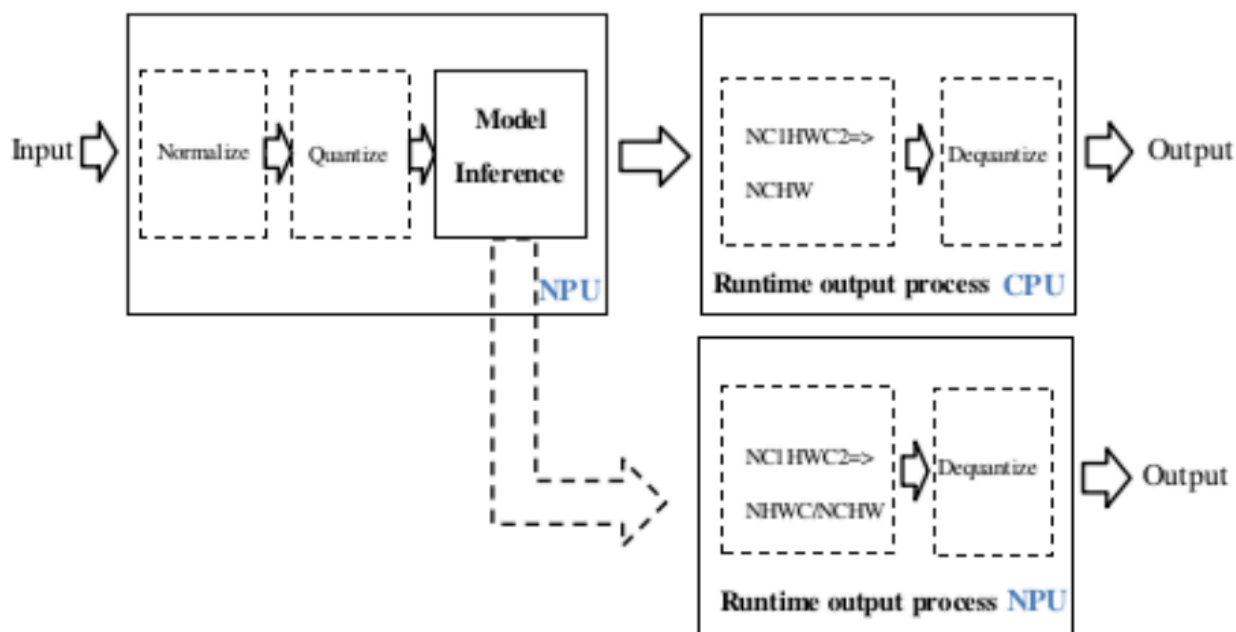
在推理RKNN模型时，原始数据要经过**输入处理、NPU运行模型、输出处理**三大流程。目前根据不同模型输入格式和量化方式，接口内部会存在通用API和零拷贝API两种处理流程，如下图所示，两组API的主要区别在于，**通用接口每次更新帧数据，需要将外部模块分配的数据拷贝到NPU运行时的输入内存，而零拷贝流程的接口会直接使用预先分配的内存**（包括NPU运行时创建的或外部其他框架创建的，比如DRM框架），减少了内存拷贝的花销，性能更优，带宽更少。当用户输入数据**只有虚拟地址**时，只能使用**通用API接口**；当用户输入数据有**物理地址或fd**时，**两组接口都可以使用。通用API和零拷贝API不能混合调用。**

通用API的数据处理流程



通用API的流程如图所示。对于数据的归一化、量化、数据排布格式转换、反量化在CPU上运行（在符合零拷贝输入要求的情况下，归一化和量化会运行在NPU上，但输入数据仍需要用CPU拷贝到模型的输入buffer上），模型本身的推理在NPU上运行。

零拷贝API数据处理流程



零拷贝API的流程如图所示。优化了通用API的数据处理流程，零拷贝API归一化、量化和模型推理都会NPU上运行，NPU输出的数据排布格式和反量化过程在CPU或者NPU上运行。零拷贝API对于输入数据流程的处理效率会比通用API高。

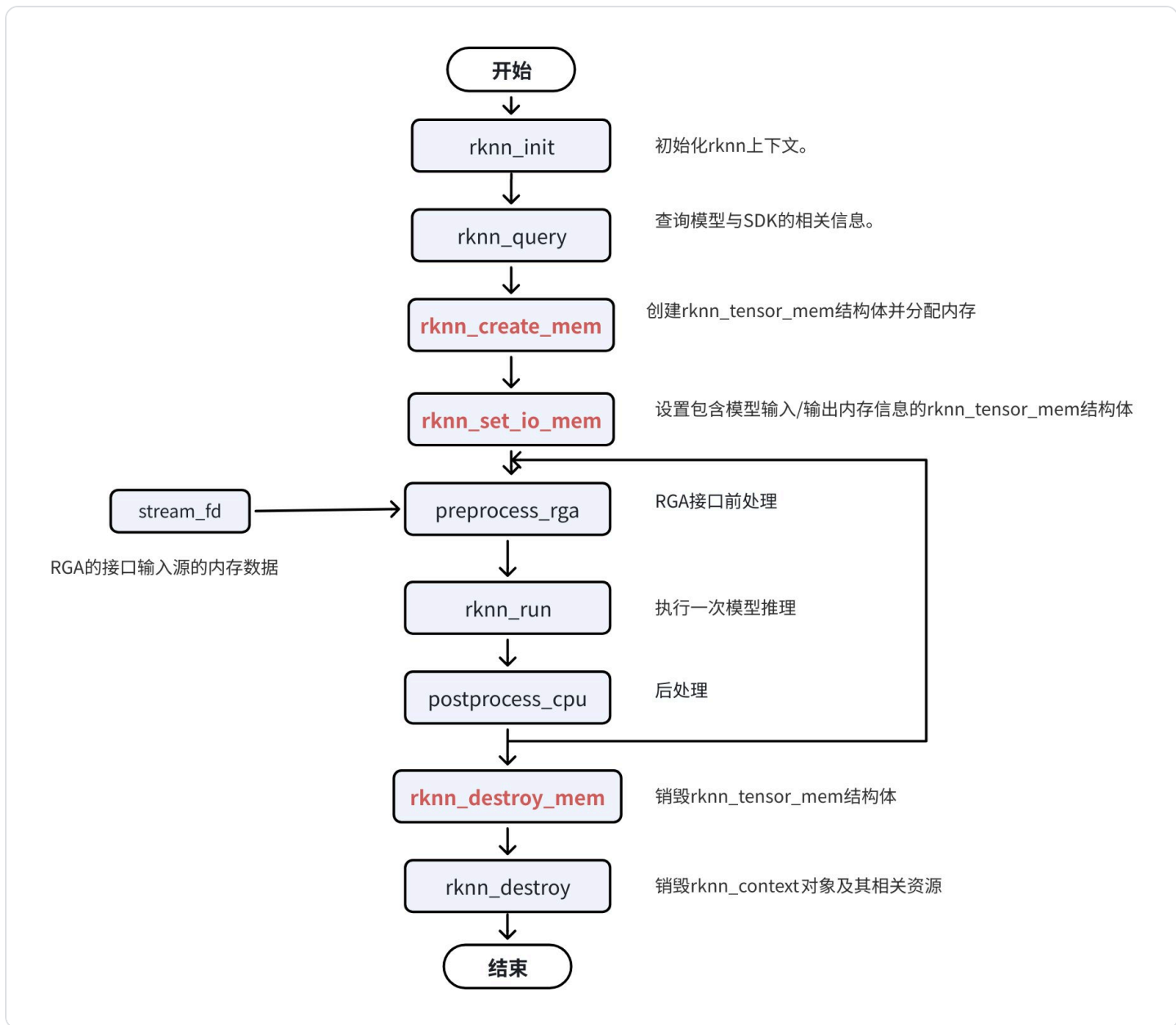
2. 零拷贝C API调用流程

零拷贝API接口使用 **rknn_tensor_memory** 结构体，需要在推理前创建并设置该结构体，并在推理后读取该结

构体中的内存信息。根据用户是否需要自行分配模型的模块内存（输入/输出/权重/中间结果）和内存表示

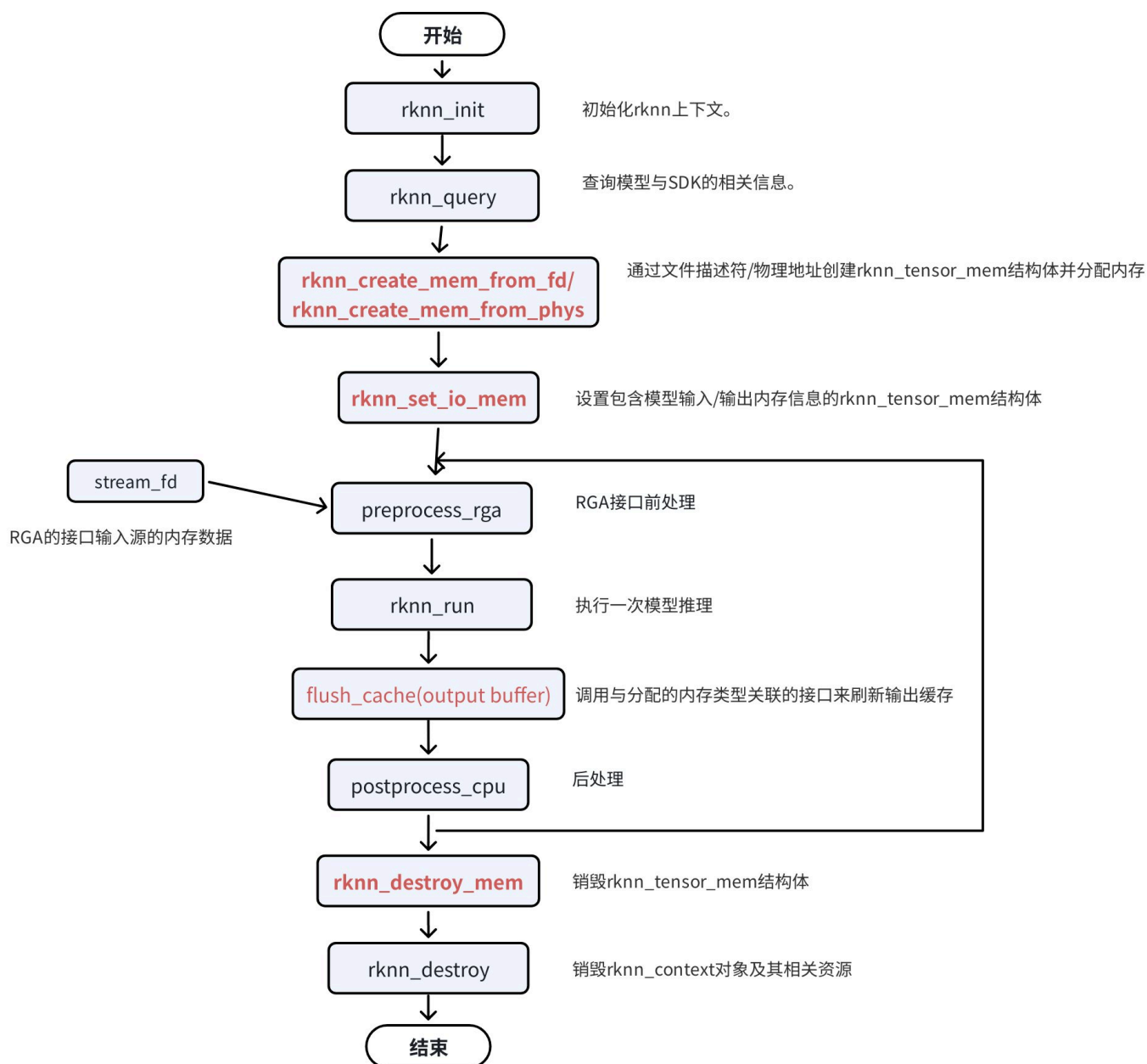
方式（文件描述符/物理地址等）差异，有下列三种典型的零拷贝调用流程。

2.1 输入/输出内存由运行时分配



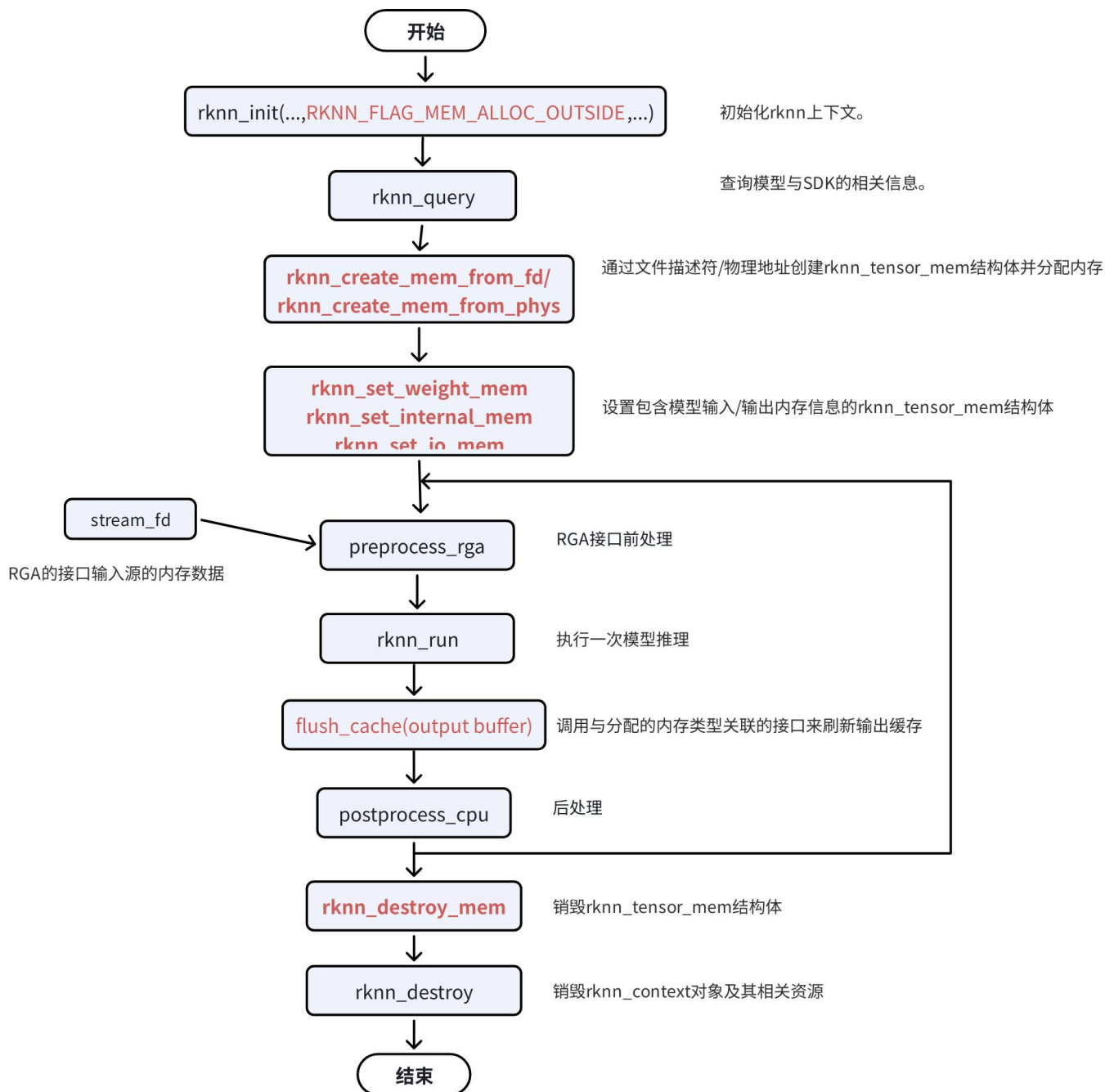
如图所示，输入/输出内存由运行时分配调用的是 `rknn_create_mem()` 接口创建 `rknn_tensor_memory` 结构体，`rknn_set_io_mem()` 设置输入输出 `rknn_tensor_memory` 结构体。`rknn_create_mem()` 接口创建的输入/输出内存信息结构体包含了文件描述符成员和物理地址，RGA的接口使用到NPU分配的内存信息，`preprocess_rga()` 表示RGA的接口，`stream_fd`表示RGA的接口输入源的内存数据，`postprocess_cpu()` 表示后处理的CPU实现。

2.2 输入/输出内存由外部分配



如图所示，输入/输出内存由外部分配调用的是 `rknn_create_mem_from_fd()` / `rknn_create_mem_from_phys()` 接口创建 `rknn_tensor_memory` 结构体，`rknn_set_io_mem()` 设置输入输出 `rknn_tensor_memory` 结构体。`flush_cache` 表示用户需要调用与分配的内存类型关联的接口来刷新输出缓存。

2.3 输入/输出/权重/中间结果内存由外部分配



如图所示，输入/输出/权重/中间结果内存由外部分配调用的是 rknn_create_mem_from_fd() / rknn_create_mem_from_phys() 接口创建 rknn_tensor_memory 结构体，rknn_set_io_mem() 设置输入输出 rknn_tensor_memory 结构体，rknn_set_weight_mem() / rknn_set_internal_mem() 设置权重/中间结果 rknn_tensor_memory 结构体。

RKNN_FLAG_MEM_ALLOC_OUTSIDE：用于表示模型输入、输出、权重、中间tensor内存全部由用户

分配，它主要有两方面的作用：

- 1.所有内存均是用户自行分配，便于对整个系统内存进行统筹安排。
- 2.用于内存复用，特别是针对RV1103/RV1106/RV1103B/RV1106B/RK2118这种内存极为紧张的情况。

3. C API零拷贝的用法

以输入/输出内存由运行时分配为例，用法如下：

3.1 rknn_query()

输入：

用RKNN_QUERY_NATIVE_INPUT_ATTR查询相关的属性（注意，不是RKNN_QUERY_INPUT_ATTR）。当查询出来的fmt（或者称为layout）不同时，需要提前处理的方式也不一样。该方式查询出来的是输入硬件效率最优的layout和type。

rknn_query() 输入的情况如下：

- a. 当layout为 RKNN_TENSOR_NCHW 时，这种情况一般输入是4维，并且数据类型为bool或者int64，当传数据给NPU时，也需要按照NCHW格式排列给NPU。
- b. 当layout为 RKNN_TENSOR_NHWC 时，这种情况一般输入是4维，并且数据类型为float32/float16/int8/uint8，同时，输入通道数是1、3、4。当传数据给NPU时，也需要按照 NHWC 格式排列给NPU。需要注意的是当 pass_through=1 时，width可能需要做stride对齐，具体取决于查询出来的 w_stride的值。
- c. 当layout为 RKNN_TENSOR_NC1HWC2 时，这种情况一般输入是4维，并且数据类型为float16/int8，同时，输入通道数不是1、3、4。当 pass_through=0 时，输入数据按照 NHWC 格式排列，接口内部会进行NHWC 到 NC1HWC2 的 CPU 转换；当 pass_through=1 时，输入数据按照 NC1HWC2 格式排列，用户外部需转换好。
- d. 当layout为 RKNN_TENSOR_UNDEFINED 时，这种情况一般输入不是4维，当传数据给NPU时，需要按照 ONNX模型输入格式传给NPU。NPU不做任何的mean/std处理以及layout转换。

如果用户需要的输入配置不同于查询接口获取的 rknn_tensor_attr 结构体，可以对 rknn_tensor_attr 结构体进行对应修改，目前支持的可修改的输入数据类型如表所示。特别注意：如果查询的数据类型是 uint8，用户想传入float32类型，则 rknn_tensor_attr 结构体的size要修改成原size的四倍，同时其中的数据类型要修改成RKNN_TENSOR_FLOAT32。用该方式修改后硬件效率就不是最优了，接口内部会调用CPU进行数据类型转换。

	rknn_query 得到的模型数据类型						
用户接口修改的数据类型		bool	int8	float16	int16	int32	Int64
	bool	Y					
	int8		Y				
	uint8		Y	Y			
	float32		Y	Y			
	float16			Y			
	int16				Y		
	int32					Y	
	Int64						Y

输出:

用RKNN_QUERY_NATIVE_OUTPUT_ATTR查询相关的属性(注,不是RKNN_QUERY_OUTPUT_ATTR) . 当查询出来的fmt（或者称为layout）不同时，需要后处理的方式也不一样。该方式查询出来的是输出硬件效率最优的layout和type。

当输出是4维且用户需要 NHWC layout的四维输出时, 可以用RKNN_QUERY_NATIVE_NHWC_OUTPUT_ATTR 查询相关的属性。该方式可以直接获得NHWC layout的输出。

rknn_query() 输出的情况如下:

- 当layout为 RKNN_TENSOR_NC1HWC2 时，这种情况一般输出是4维，并且数据类型为float16/int8。当用户需要 NCHW layout时，外部需进行 NC1HWC2 到 NCHW 的layout 转换。
- 当layout为 RKNN_TENSOR_UNDEFINED 时，这种情况一般输出非4维，并且数据类型为float16/int8。用户外部无需进行layout转换。
- 当layout为 RKNN_TENSOR_NCHW 时，这种情况一般输出是4维, 并且数据类型为float16/int8。用户外部 无需进行layout转换。
- 当layout为 RKNN_TENSOR_NHWC 时，这种情况一般输出是4维, 并且数据类型为float16/int8。这种情况 一般是用户调用 RKNN_QUERY_NATIVE_NHWC_OUTPUT_ATTR 接口查询出来的layout。

如果用户需要的输出配置不同于查询接口获取的 rknn_tensor_attr 结构体，可以对 rknn_tensor_attr 结构体进行对应修改，可修改的配置信息如所示，特别注意：如果查询输出的数据类型是int8，用户想获取成float32类型输出，则 rknn_tensor_attr 结构体的size要修改成原size的四倍，同时其中的数据

类型要修改成 RKNN_TENSOR_FLOAT32。用该方式修改后硬件效率就不是最优了，接口内部会调用 CPU进行数据类型转换。

输出可修改的输入数据类型表

	rknn_query 得到的模型数据类型						
用户接口修改的数据类型		bool	int8	float16	int16	int32	Int64
	bool	Y					
	int8		Y				
	uint8						
	float32		Y	Y			
	float16			Y			
	int16				Y		
	int32					Y	
	Int64						Y

输出可修改的layout类型表

	rknn_query 查询得到的模型layout类型				
		NC1HWC2	NCHW	NHWC	UNDEFINE
用户接口设置的layout类型	NC1HWC2	Y			

	rknn_query 查询得到的模型layout类型				
	NCHW	Y	Y		
	NHWC			Y	
	UNDEFINE				Y

模型类型	输出数据类型	输出维度	可支持output layout
int8模型	int8/float16/float32	4维	NCHW/NC1HWC2/NHWC
		非4维	UNDEFINE
float16模型	float16/float32	4维	NCHW/NC1HWC2/NHWC
		非4维	UNDEFINE

3.2 rknn_create_mem

零拷贝API接口使用 rknn_tensor_memory 结构体，需要在推理前创建并设置该结构体，并在推理后读取该结 构体中的内存信息。当无需对 RKNN_QUERY_NATIVE_INPUT_ATTR ， RKNN_QUERY_NATIVE_OUTPUT_ATTR 查询出来的layout和type进行修改时，直接采用默认配置的 size_with_stride 创建内存大小。若修改了相应的layout和type，则需按照相应的size 创建内存大小。(例如输出的数据类型是int8，用户想获取成float32类型输出，size要修改成原size的四倍)。

3.3 rknn_set_io_mem

rknn_set_io_mem() 用于设置包含模型输入/输出内存信息的 rknn_tensor_mem 结构体，和 rknn_init() 类似只要在最开始调用一次，后面反复执行 rknn_run() 即可。

4. 环境运行搭建

这里参考前面第8节，快速体验rknn_demo那篇，就不详细写了。

1.设置工具链

代码块

```
1 export GCC_COMPILER=/opt/tool_chain/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu
```

2.构建

代码块

```
1 ./build-linux.sh -t rk3588 -a aarch64 -b Release
```

3.将编译出来的install目录下的文件拷贝到开发板上

```
代码块 scp -r rknn_zero_copy_test_Linux/ root@192.168.137.142:/userdata
```

4.开发板执行

代码块

```
1 cd rknn_zero_copy_test_Linux/
2
3 export LD_LIBRARY_PATH=./lib
4
5 ./rknn_zero_copy_test ./model/RK3588/mobilenet_v1.rknn ./model/dog_224x224.jpg
```

5. 代码测试

17_rknpu_zero_copy

代码块

```
1 #include "opencv2/core/core.hpp"
2 #include "opencv2/imgcodecs.hpp"
3 #include "opencv2/imgproc.hpp"
4 #include "rknn_api.h"
5
6 #include <float.h>
7 #include <stdio.h>
8 #include <stdlib.h>
9 #include <string.h>
10 #include <sys/time.h>
11
12 using namespace std;
13 using namespace cv;
14
15 /*-----
16                      Functions
17 -----*/
18 // 获取当前系统时间，返回微秒级时间戳，用于后续计算推理耗时
19 static inline int64_t getCurrentTimeUs()
20 {
21     struct timeval tv;
22     gettimeofday(&tv, NULL);
23     return tv.tv_sec * 1000000 + tv.tv_usec;
24 }
25
26 // 从模型输出的概率数组中提取 TopN 个最高概率及其对应的类别索引
```

```

27 static int rknn_GetTopN(float* pfProb, float* pfMaxProb, uint32_t* pMaxClass,
uint32_t outputCount, uint32_t topNum)
28 {
29     uint32_t i, j;
30     uint32_t top_count = outputCount > topNum ? topNum : outputCount;
31
32     for (i = 0; i < topNum; ++i) {
33         pfMaxProb[i] = -FLT_MAX;
34         pMaxClass[i] = -1;
35     }
36
37     for (j = 0; j < top_count; j++) {
38         for (i = 0; i < outputCount; i++) {
39             if ((i == *(pMaxClass + 0)) || (i == *(pMaxClass + 1)) || (i == *
(pMaxClass + 2)) || (i == *(pMaxClass + 3)) ||
40                 (i == *(pMaxClass + 4))) {
41                 continue;
42             }
43
44             if (pfProb[i] > *(pfMaxProb + j)) {
45                 *(pfMaxProb + j) = pfProb[i];
46                 *(pMaxClass + j) = i;
47             }
48         }
49     }
50
51     return 1;
52 }
53
54 // 打印张量属性信息
55 static void dump_tensor_attr(rknn_tensor_attr* attr)
56 {
57     printf("  index=%d, name=%s, n_dims=%d, dims=[%d, %d, %d, %d], n_elems=%d,
size=%d, fmt=%s, type=%s, qnt_type=%s, "
58         "zp=%d, scale=%f\n",
59         attr->index, attr->name, attr->n_dims, attr->dims[0], attr->dims[1],
attr->dims[2], attr->dims[3],
60         attr->n_elems, attr->size, get_format_string(attr->fmt),
get_type_string(attr->type),
61         get_qnt_type_string(attr->qnt_type), attr->zp, attr->scale);
62 }
63
64 /*-----
65                               Main Functions
66 -----*/
67 int main(int argc, char* argv[])
68 {

```

```
69     if (argc < 3) {
70         printf("Usage:%s model_path input_path [loop_count]\n", argv[0]);
71         return -1;
72     }
73
74     char* model_path = argv[1];
75     char* input_path = argv[2];
76
77     int loop_count = 1;
78     if (argc > 3) {
79         loop_count = atoi(argv[3]);
80     }
81
82     rknn_context ctx = 0;
83
84     // 【零拷贝差异】初始化方式与通用API一致，但后续内存管理完全不同
85     int ret = rknn_init(&ctx, model_path, 0, 0, NULL);
86     if (ret < 0) {
87         printf("rknn_init fail! ret=%d\n", ret);
88         return -1;
89     }
90
91     // 查询 SDK 版本
92     rknn_sdk_version sdk_ver;
93     ret = rknn_query(ctx, RKNN_QUERY_SDK_VERSION, &sdk_ver, sizeof(sdk_ver));
94     if (ret != RKNN_SUCC) {
95         printf("rknn_query fail! ret=%d\n", ret);
96         return -1;
97     }
98     printf("rknn_api/rknnrt version: %s, driver version: %s\n",
99           sdk_ver.api_version, sdk_ver.drv_version);
100
101     // 查询输入输出数量
102     rknn_input_output_num io_num;
103     ret = rknn_query(ctx, RKNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
104     if (ret != RKNN_SUCC) {
105         printf("rknn_query fail! ret=%d\n", ret);
106         return -1;
107     }
108     printf("model input num: %d, output num: %d\n", io_num.n_input,
109           io_num.n_output);
110
111     // 遍历并打印所有输入张量的属性信息
112     printf("input tensors:\n");
113     rknn_tensor_attr input_attrs[io_num.n_input];
114     memset(input_attrs, 0, io_num.n_input * sizeof(rknn_tensor_attr));
115     for (uint32_t i = 0; i < io_num.n_input; i++) {
```

```

114     input_attrs[i].index = i;
115     ret = rknn_query(ctx, RKNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
sizeof(rknn_tensor_attr));
116     if (ret < 0) {
117         printf("rknn_init error! ret=%d\n", ret);
118         return -1;
119     }
120     dump_tensor_attr(&input_attrs[i]);
121 }
122
123 // 遍历并打印所有输出张量的属性信息
124 printf("output tensors:\n");
125 rknn_tensor_attr output_attrs[io_num.n_output];
126 memset(output_attrs, 0, io_num.n_output * sizeof(rknn_tensor_attr));
127 for (uint32_t i = 0; i < io_num.n_output; i++) {
128     output_attrs[i].index = i;
129     ret = rknn_query(ctx, RKNN_QUERY_OUTPUT_ATTR, &(output_attrs[i]),
sizeof(rknn_tensor_attr));
130     if (ret != RKNN_SUCC) {
131         printf("rknn_query fail! ret=%d\n", ret);
132         return -1;
133     }
134     dump_tensor_attr(&output_attrs[i]);
135 }
136
137 // 查询自定义字符串
138 rknn_custom_string custom_string;
139 ret = rknn_query(ctx, RKNN_QUERY_CUSTOM_STRING, &custom_string,
sizeof(custom_string));
140 if (ret != RKNN_SUCC) {
141     printf("rknn_query fail! ret=%d\n", ret);
142     return -1;
143 }
144 printf("custom string: %s\n", custom_string.string);
145
146 // 定义输入数据的默认类型和内存布局
147 unsigned char*    input_data    = NULL;
148 rknn_tensor_type  input_type    = RKNN_TENSOR_UINT8;
149 rknn_tensor_format input_layout = RKNN_TENSOR_NHWC;
150
151 // 根据模型要求的输入格式，获取目标宽高和通道数
152 int req_height = 0;
153 int req_width  = 0;
154 int req_channel = 0;
155
156 switch (input_attrs[0].fmt) {
157 case RKNN_TENSOR_NHWC:

```

```

158     req_height  = input_attrs[0].dims[1];
159     req_width   = input_attrs[0].dims[2];
160     req_channel  = input_attrs[0].dims[3];
161     break;
162 case RKNN_TENSOR_NCHW:
163     req_height  = input_attrs[0].dims[2];
164     req_width   = input_attrs[0].dims[3];
165     req_channel  = input_attrs[0].dims[1];
166     break;
167 default:
168     printf("meet unsupported layout\n");
169     return -1;
170 }
171
172 // 使用 OpenCV 读取原始图片
173 int height  = 0;
174 int width   = 0;
175 int channel  = 0;
176
177 cv::Mat orig_img = imread(input_path, cv::IMREAD_COLOR);
178 if (!orig_img.data) {
179     printf("cv::imread %s fail!\n", input_path);
180     return -1;
181 }
182
183 // BGR 转 RGB
184 cv::Mat orig_img_rgb;
185 cv::cvtColor(orig_img, orig_img_rgb, cv::COLOR_BGR2RGB);
186
187 // 尺寸缩放
188 cv::Mat img = orig_img_rgb.clone();
189 if (orig_img.cols != req_width || orig_img.rows != req_height) {
190     printf("resize %d %d to %d %d\n", orig_img.cols, orig_img.rows, req_width,
req_height);
191     cv::resize(orig_img_rgb, img, cv::Size(req_width, req_height), 0, 0,
cv::INTER_LINEAR);
192 }
193 input_data = img.data;
194 if (!input_data) {
195     return -1;
196 }
197
198 // 【零拷贝差异 1: 手动创建 NPU 物理内存】
199 // 通用 API 中不需要这一步，数据由 rknn_run 内部自动搬运。
200 // 零拷贝模式下，必须显式调用 rknn_create_mem 在 DDR 保留区申请 NPU 可访问的物理连续
内存。
201 rknn_tensor_mem* input_mems[1];

```

```

202
203 // 【零拷贝差异 2: 强制指定 NHWC 格式与 UINT8 类型】
204 // 零拷贝模式下, NPU 仅支持 NHWC 格式的 uint8 输入。
205 // NPU 硬件会自动融合归一化和量化操作, 因此输入必须是 uint8。
206 input_attrs[0].type = input_type;
207 input_attrs[0].fmt = input_layout;
208
209 // 【零拷贝差异 3: 按带步长的尺寸分配内存】
210 // 注意这里使用的是 size_with_stride 而不是普通 size。
211 // NPU 对内存对齐有严格要求, 每一行像素末尾可能存在 Padding。
212 input_mems[0] = rknn_create_mem(ctx, input_attrs[0].size_with_stride);
213
214 // 【零拷贝差异 4: 手动处理步长对齐拷贝】
215 // 这是最容易踩坑的地方! 如果图像宽度小于步长, 必须逐行拷贝, 跳过填充区域。
216 // 通用 API 会在底层自动处理这个对齐, 但零拷贝必须由开发者手动完成。
217 width      = input_attrs[0].dims[2];
218 int stride = input_attrs[0].w_stride;
219
220 if (width == stride) {
221     memcpy(input_mems[0]->virt_addr, input_data, width *
input_attrs[0].dims[1] * input_attrs[0].dims[3]);
222 } else {
223     int height = input_attrs[0].dims[1];
224     int channel = input_attrs[0].dims[3];
225     uint8_t* src_ptr = input_data;
226     uint8_t* dst_ptr = (uint8_t*)input_mems[0]->virt_addr;
227     int src_wc_elems = width * channel;
228     int dst_wc_elems = stride * channel;
229     for (int h = 0; h < height; ++h) {
230         memcpy(dst_ptr, src_ptr, src_wc_elems);
231         src_ptr += src_wc_elems;
232         dst_ptr += dst_wc_elems;
233     }
234 }
235
236 // 【零拷贝差异 5: 为输出也手动分配内存】
237 // 通用 API 的输出由框架自动管理, 零拷贝需手动分配。
238 // 这里分配 float32 是为了方便 CPU 直接读取结果进行 Top5 计算。
239 rknn_tensor_mem* output_mems[io_num.n_output];
240 for (uint32_t i = 0; i < io_num.n_output; ++i) {
241     int output_size = output_attrs[i].n_elems * sizeof(float);
242     output_mems[i] = rknn_create_mem(ctx, output_size);
243 }
244
245 // 【零拷贝差异 6: 绑定 IO 内存到上下文】
246 // 通用 API 使用 rknn_inputs_set / rknn_outputs_get 传递数据指针。
247 // 零拷贝模式使用 rknn_set_io_mem 将刚才手动分配的物理内存句柄绑定给 NPU。

```

```

248     ret = rknn_set_io_mem(ctx, input_mems[0], &input_attrs[0]);
249     if (ret < 0) {
250         printf("rknn_set_io_mem fail! ret=%d\n", ret);
251         return -1;
252     }
253
254     for (uint32_t i = 0; i < io_num.n_output; ++i) {
255         output_attrs[i].type = RKNN_TENSOR_FLOAT32;
256         ret = rknn_set_io_mem(ctx, output_mems[i], &output_attrs[i]);
257         if (ret < 0) {
258             printf("rknn_set_io_mem fail! ret=%d\n", ret);
259             return -1;
260         }
261     }
262
263     // 开始性能测试循环
264     printf("Begin perf ...\n");
265     for (int i = 0; i < loop_count; ++i) {
266         int64_t start_us = getCurrentTimeUs();
267
268         // 【零拷贝差异 7: rknn_run 无数据传输开销】
269         // 此时数据已经在 NPU 物理内存中, run 只触发计算, 没有 memcpy 延迟。
270         ret = rknn_run(ctx, NULL);
271
272         int64_t elapse_us = getCurrentTimeUs() - start_us;
273         if (ret < 0) {
274             printf("rknn run error %d\n", ret);
275             return -1;
276         }
277         printf("%4d: Elapse Time = %.2fms, FPS = %.2f\n", i, elapse_us / 1000.f,
1000.f * 1000.f / elapse_us);
278     }
279
280     // 解析输出结果
281     uint32_t topNum = 5;
282     for (uint32_t i = 0; i < io_num.n_output; i++) {
283         uint32_t MaxClass[topNum];
284         float fMaxProb[topNum];
285
286         // 【零拷贝差异 8: 直接从绑定的输出内存读取结果】
287         // 无需调用 rknn_outputs_get, 结果已经在我们手动分配的 output_mems 中。
288         float* buffer = (float*)output_mems[i]->virt_addr;
289         uint32_t sz = output_attrs[i].n_elems;
290         int top_count = sz > topNum ? topNum : sz;
291
292         rknn_GetTopN(buffer, fMaxProb, MaxClass, sz, topNum);
293

```



```

294     printf("---- Top%d ----\n", top_count);
295     for (int j = 0; j < top_count; j++) {
296         printf("%.6f - %d\n", fMaxProb[j], MaxClass[j]);
297     }
298 }
299
300 // 【零拷贝差异 9: 手动销毁内存】
301 // 通用 API 无需手动释放 IO 内存。零拷贝模式下, 谁申请谁释放, 防止 DDR 内存泄漏。
302 rknn_destroy_mem(ctx, input_mems[0]);
303 for (uint32_t i = 0; i < io_num.n_output; ++i) {
304     rknn_destroy_mem(ctx, output_mems[i]);
305 }
306
307 // 销毁 RKNN 上下文
308 rknn_destroy(ctx);
309 return 0;
310 }

```

查看结果

```

root@topeet:/userdata/rknn_zero_copy_test_Linux# ls
lib  model  rknn_zero_copy_test
root@topeet:/userdata/rknn_zero_copy_test_Linux# ./rknn_zero_copy_test ./model/RK3588/mobilenet_v1.rknn ./model/dog_224x224.jpg
rknn_api/rknnrt version: 2.3.2 (429f97ae6b@2025-04-09T09:09:27), driver version: 0.9.8
model input num: 1, output num: 1
input tensors:
  index=0, name=input, n_dims=4, dims=[1, 224, 224, 3], n_elems=150528, size=150528, fmt=NHWC, type=INT8, qnt_type=AFFINE,
  zp=0, scale=0.007812
output tensors:
  index=0, name=MobilenetV1/Predictions/Reshape_1, n_dims=2, dims=[1, 1001, 0, 0], n_elems=1001, size=2002, fmt=UNDEFINED,
  type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000
custom string:
Begin perf ...
  0: Elapse Time = 3.06ms, FPS = 327.33
---- Top5 ----
0.878906 - 156
0.059814 - 155
0.004082 - 205
0.003651 - 284
0.000215 - 285
root@topeet:/userdata/rknn_zero_copy_test_Linux#

```